Take a website change request from Asana to live

How a change request becomes a live client site without ever breaking the live site. If you remember one thing: **never work directly on main** — main is the branch that *is* the live site. A branch is your own private copy of the site, and nothing on it reaches the live site until someone deliberately merges it.

SOP ID	HGM-SOP-WEB-002	EFFECTIVE	2026-09-01
VERSION	1.5	NEXT REVIEW	2027-03-01
OWNER	Web & Content Specialist (Kyle Zinger)	TRIGGER	Every website change request
APPROVED BY	Operations Manager (Gillian)	DURATION	~45 minutes for a small edit

01 PURPOSE AND SCOPE

Purpose

Takes a website change request from an Asana task all the way to the live client site without ever putting the live site at risk. Every step that touches the terminal (the panel in VS Code where you type commands) offers two or three routes, so nobody has to type a command they do not understand.

In scope

Content and code changes to HGM's Next.js client sites hosted on Netlify, requested through Asana and made in VS Code with Claude Code.

Out of scope

Building a new site from scratch, hosting or domain (DNS) changes, and anything on a site that is not one of the Next.js client repositories (code stores) in the `hgmwebteam` GitHub organization.

ALWAYS TRUE

Four standing rules

True on every job.

1

Never work directly on main

Work on a branch. Nothing on it reaches the live site until someone merges it.

2

Never send a credential

Not in Google Chat, email or Asana, and never commit `.env.local` (the site's file of keys; see HGM-SOP-WEB-001). Point people at 1Password.

3

Approval is routed, not sent to one person

Who signs off depends on the change (the table at task 6.1), and silence is not a yes.

4

Show the preview, not your laptop

Your `localhost` copy proves nothing to anyone else. Check your change on the Deploy Preview URL (Netlify's working copy of your pull request) first.

02 PREREQUISITES

Phase 1 is once per machine, per site. After that, every job starts at Phase 2.

1 **VS Code**

Download from `code.visualstudio.com`.

2 **Node.js**

Version 20 or newer.

3 **Claude Code**

Install from the VS Code Extensions panel, then sign in.

4 **1Password**

Desktop app, signed in to the HGM account with access to the client's vault.

5 **GitHub access**

Membership of the `hgmwebteam` organization. Request from a developer in the **Web Team** Google Chat group.

6 No Netlify account needed

Netlify already watches these repositories and builds the preview itself when a pull request (the GitHub page where a branch is reviewed and merged) is opened. You never log in to Netlify or run a deploy command.

Recommended default

New to this? Use the Claude option everywhere. It checks things the raw commands do not, and the credit cost of the mechanical steps is negligible.

Comfortable in a terminal? Run tasks 2.1, 4.2 and 6.2 yourself — they are quick and predictable. Still use the Claude option for **task 5.1**, because it confirms your change is actually on the Deploy Preview URL, which is the mistake people make most often.

03 PROCEDURE

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1
Set up a machine
Tasks 1.1–1.9 · first time on this computer or site

PHASE 2
Start a new request
Tasks 2.1–2.2 · clean branch, gather the request

PHASE 3
Plan before building
Tasks 3.1–3.2 · get a plan, not code

PHASE 4
Build and check
Tasks 4.1–4.2 · build it, then check it locally

PHASE 5
Put it on a preview
Tasks 5.1–5.2 · pull request and Deploy Preview

PHASE 6
Approve and go live
Tasks 6.1–6.3 · route, merge, verify

Phase 1 — Set up a machine

Once per machine, per site. After this, every job starts at Phase 2.

1.1 Install the four tools

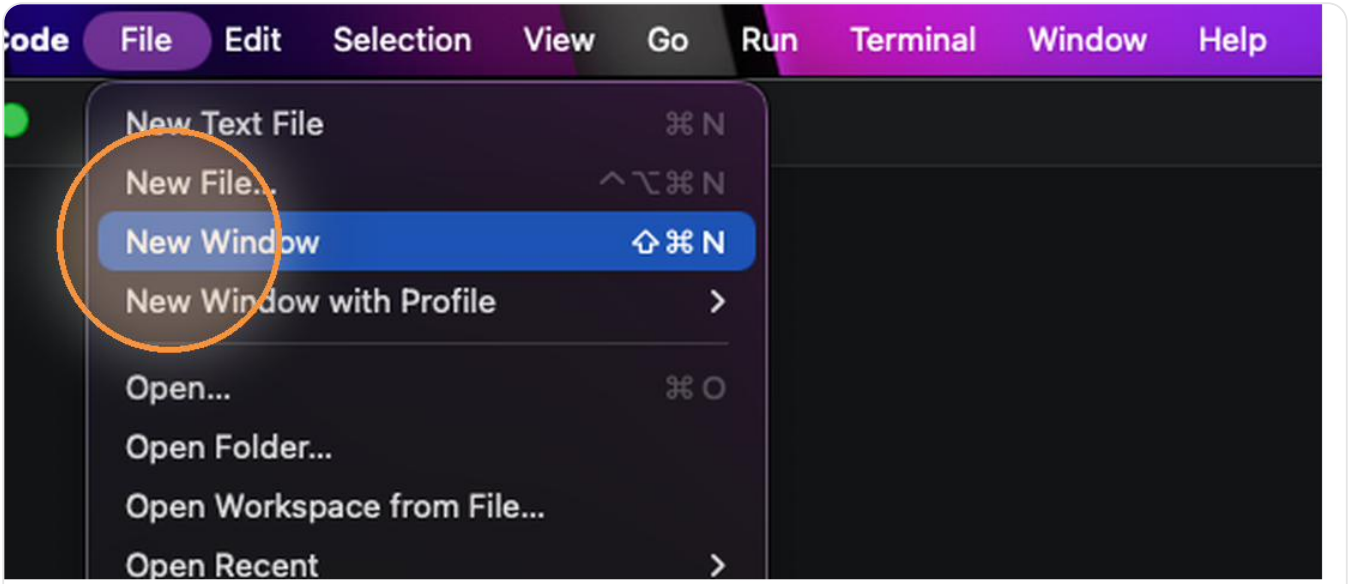
VS Code, Node.js 20 or newer, the Claude Code extension, and the 1Password desktop app. Links are in prerequisites above.

Expected result

Claude Code appears in the VS Code Extensions panel, signed in.

1.2 Open a new VS Code window

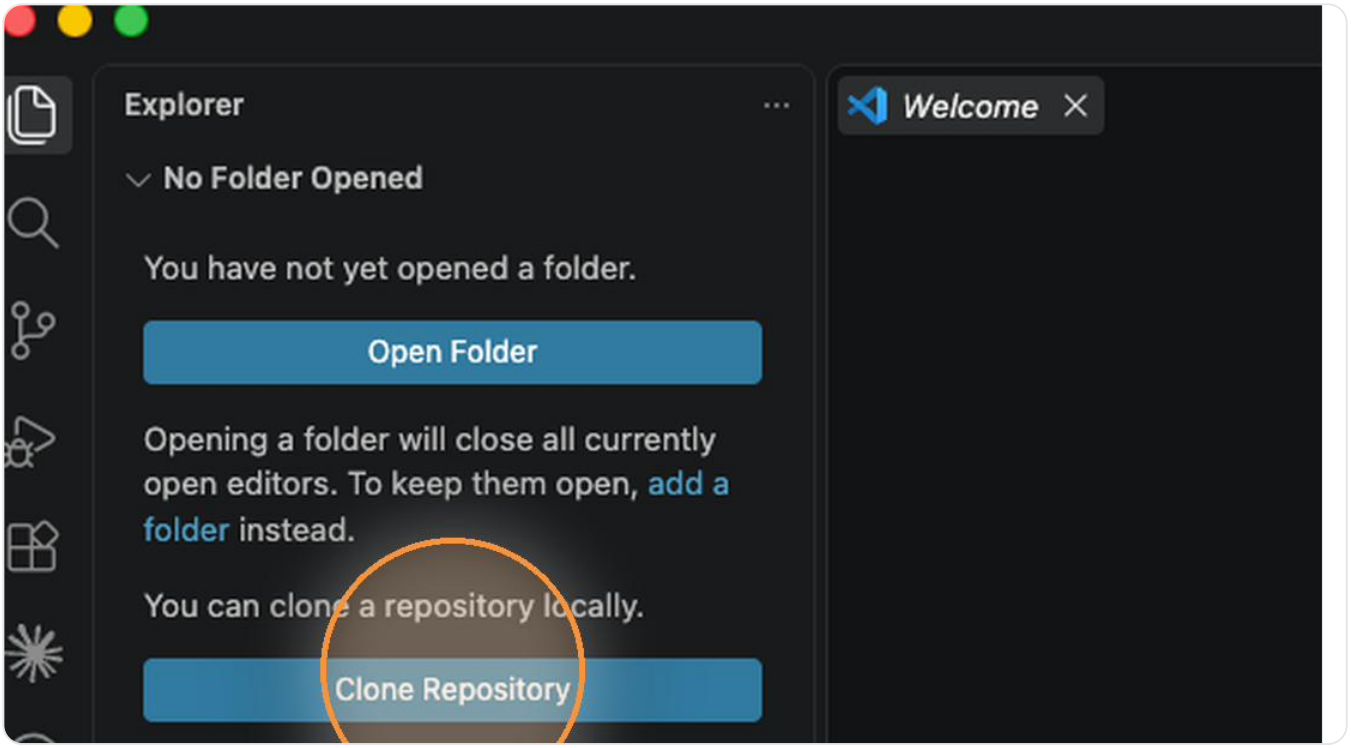
Choose **File → New Window**.



File → New Window.

1.3 Open the Explorer, then click Clone Repository

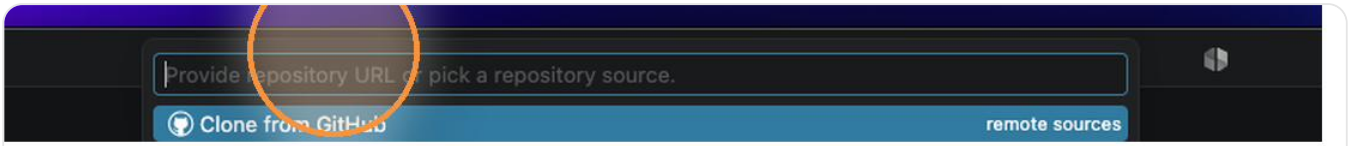
The Explorer icon is at the top of the left sidebar. With no folder open it offers **Open Folder** and **Clone Repository**.



The Explorer panel with no folder open.

1.4 Paste the repository URL and press Return

Then choose a folder to put the site in.



The repository URL prompt.

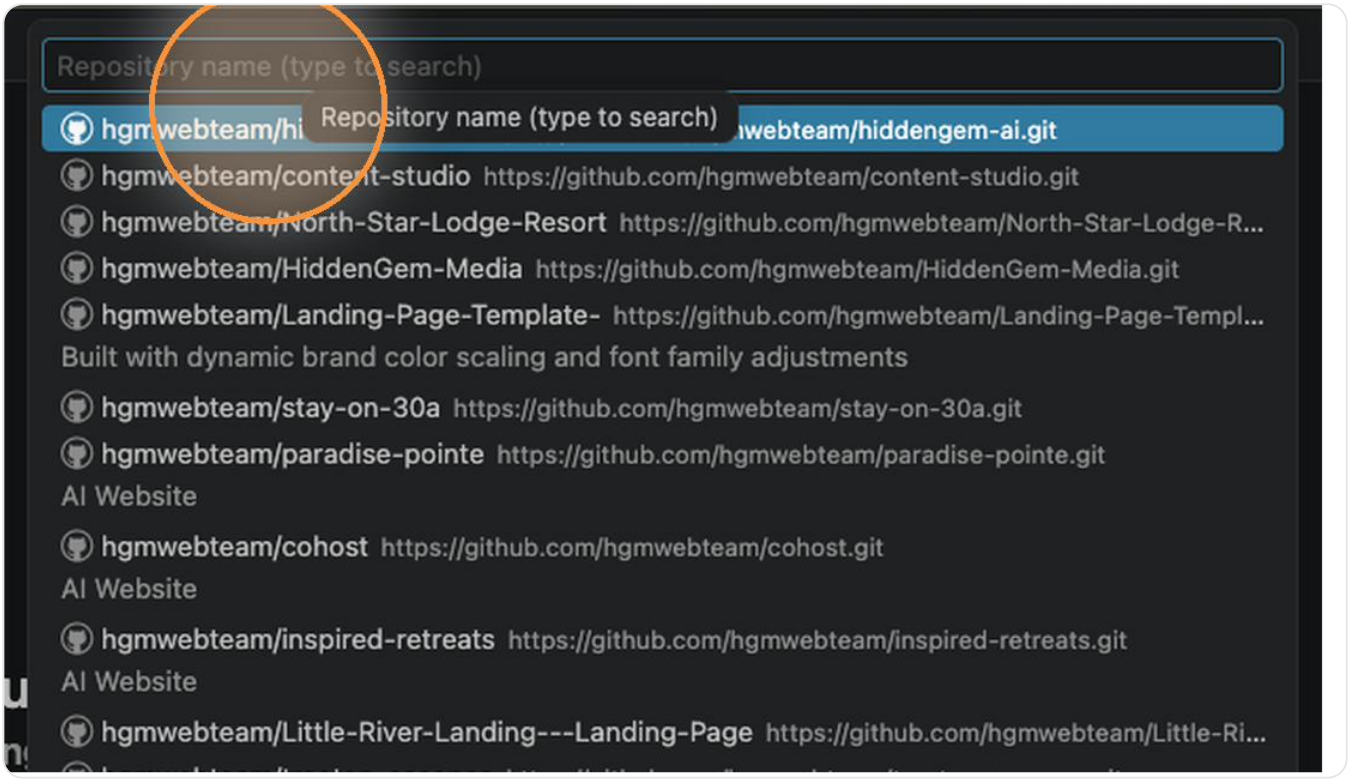
REPOSITORY URLS — ONE PER CLIENT SITE

```
https://github.com/hgmwebteam/<site-name>.git

# taberg-falls · flohom · paradise-pointe
# treetop-escapes · stay-saluda · stay-on-30a
# ridge-and-falls · cohost
```

1.5 Or pick the site from the list instead of typing

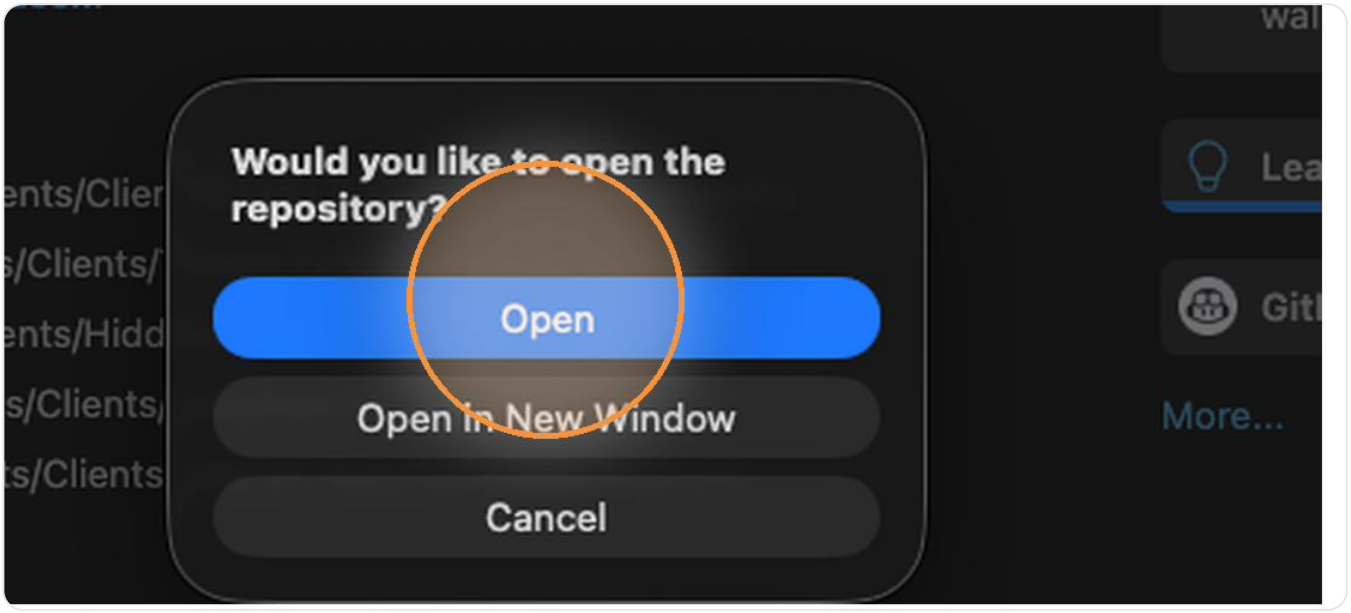
The same prompt lists every repository in the `hgmwebteam` organization.



Start typing the site name to filter the list.

1.6 Click Open when VS Code asks

It asks “Would you like to open the repository?” once the clone finishes. Choose **Open**, not Open in New Window.



Click **Open**.

If VS Code asks you to sign in to GitHub

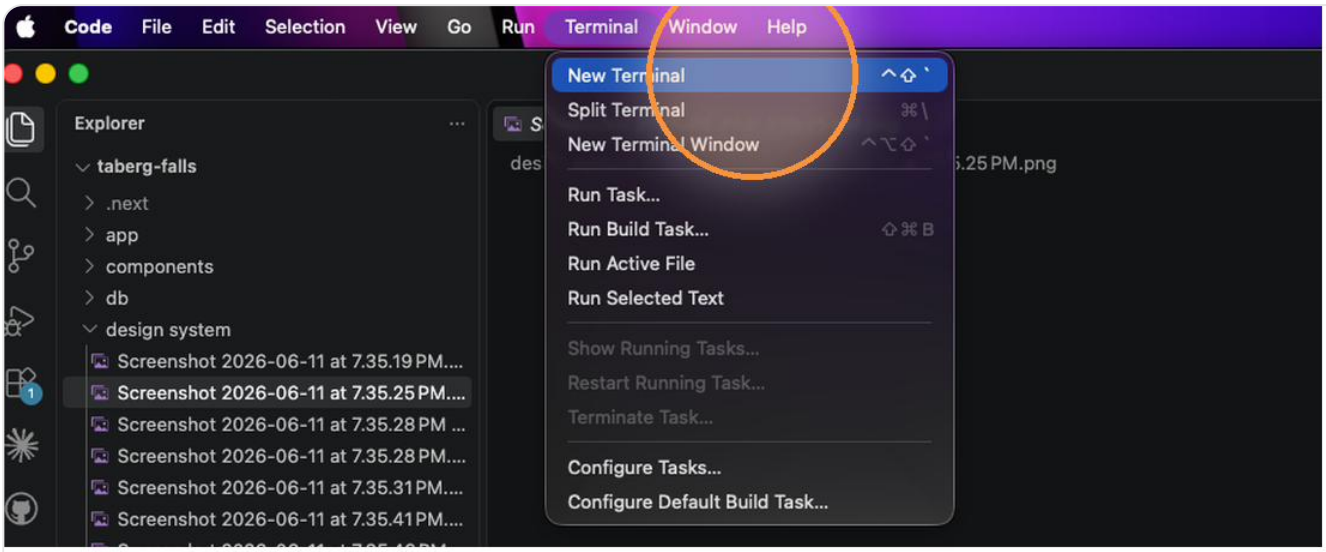
Do it. It only asks once per machine.

Expected result

The site's files appear in the Explorer panel.

1.7 Run npm install

Open a terminal with **Terminal → New Terminal** (or `Ctrl + ``) and run `npm install`. It downloads the code the site depends on.



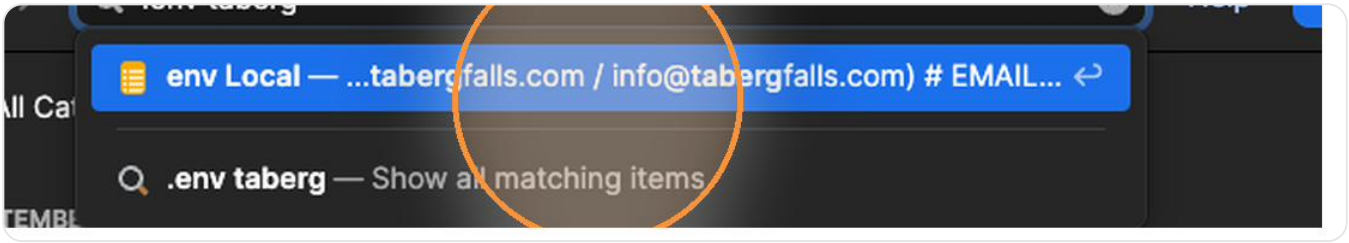
Terminal → New Terminal.

Expected result

It finishes in a minute or two. Once per site, not once per job.

1.8 Find the site's .env.local entry in 1Password

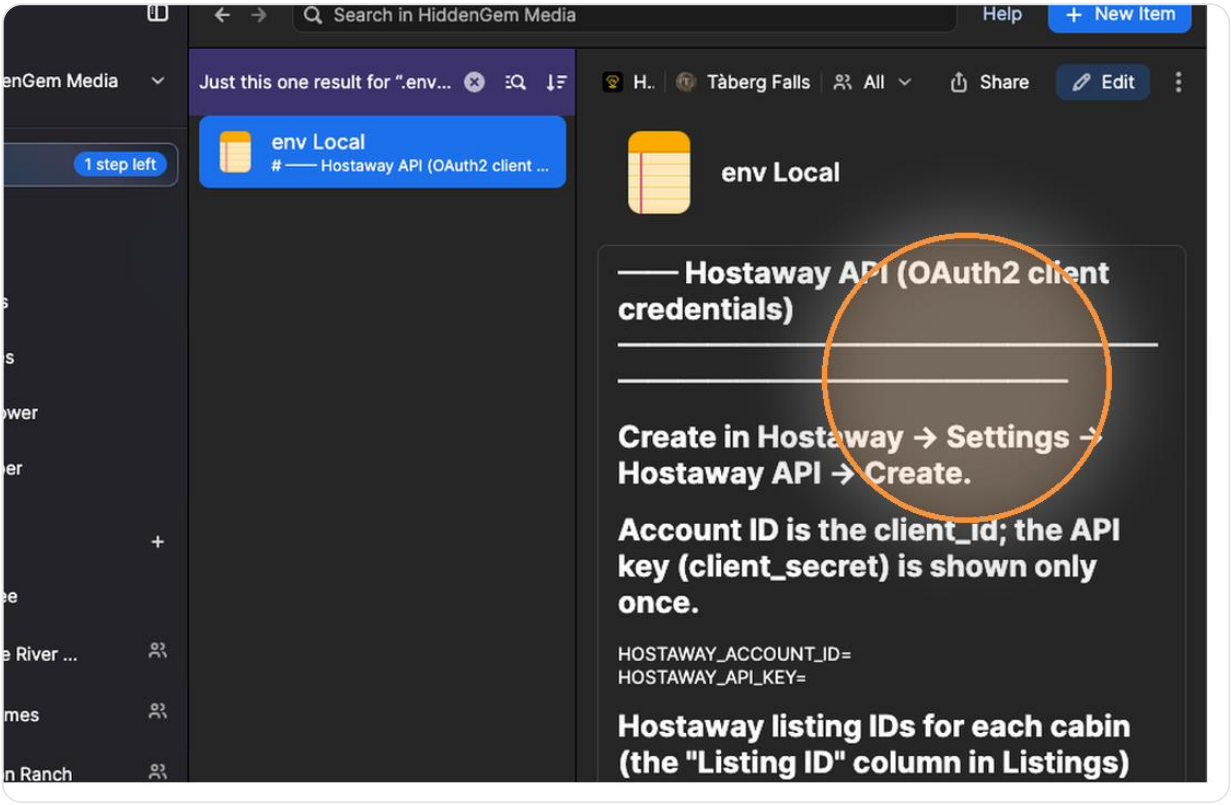
Search the site name plus `env` — for example *Taberg Falls .env.local*. This file holds the site's credentials. Without it the site still runs, but property photos and prices will not load.



Searching 1Password for the environment file.

1.9 Copy the entry, then create .env.local in VS Code

Copy the entry's **entire** contents. In VS Code create a new file called exactly `.env.local` in the top folder of the site (the project root), alongside `package.json`. Paste, and save. Create it in VS Code, not Finder — Finder hides files whose names start with a dot.



The entry. Copy all of it, comments included.

Expected result

The site runs locally with photos and prices loading.

If it is not in 1Password

Ask AnhTuan, Leshan, Brandon or Kyle — one of them will have it. Then, with as much warmth as you can manage, ask them to **put it in 1Password**, because that is where it was supposed to live all along. You are not the last person who will need it. Full procedure in HGM-SOP-WEB-001.

Never send a credential

Do not paste credentials into Google Chat, email, or an Asana comment, and do not add `.env.local` to the code repository. Git is set to ignore that file on purpose — leave it that way.

Doing Phase 1 in one go instead

Tasks 1.2 to 1.7 are the click-by-click route. Either of these does the same thing in one action.

TERMINAL / CLI — COMMAND PALETTE

```
Press Cmd + Shift + P, type: Git: Clone
Paste the same URL, choose a folder, then let VS Code open it.
Then run: npm install
```

CLAUDE PROMPT — PASTE THIS IN

```
Clone https://github.com/hgmwebteam/[site-name].git into
this folder, open it, and then run npm install. Tell me if
anything fails.
```

Phase 2 — Start a new request

Begin here for every new job.

2.1 Get onto a clean, current branch

Three things happen here: confirm nothing is half-finished, get the latest code, and make a fresh branch to work on. Git is what tracks every change to the code; a commit is one saved set of changes on your branch.

TERMINAL / CLI — TYPE IT YOURSELF

```
git checkout main
git pull
git status

# then, naming the branch after the job:
git checkout -b fix/terms-page-update
```

CLAUDE PROMPT — PASTE THIS IN

```
I'm starting a new website change request. Please get me set up:

1. Check I'm on main, that my working copy is clean, and that
   it's in sync with origin/main. If there are uncommitted
   changes I don't recognise, STOP and show me what they are.
   Do not discard anything.
2. Pull the latest main.
3. Create a branch for this job called: fix/[short-description]
   Use fix/ for corrections, feat/ for new features.
4. Confirm which branch I'm on now.

Make no other changes.
```

Stop if you see changes you do not recognise

Ask a developer before going further. That is someone's unfinished work, or your own from last time, and carrying it into a new job tangles two changes together.

Expected result

`git status` reports nothing to commit (a "clean tree"), and you are on your new branch.

2.2 Gather the whole request before opening Claude

Collect the Asana task text, the screenshots, the Loom transcript if there is one, and any follow-up messages.

Claude cannot watch a Loom

It cannot see video or hear audio, and it cannot open your Asana. Paste the **transcript text** instead. Auto-transcripts garble names badly — if a word looks like a client name, say what you think it means so Claude does not guess wrong.

Phase 3 — Plan before building

The step people skip, and the one that prevents the most rework.

3.1 Ask for a plan — not for code

Paste the whole prompt, with the request pasted into the marked block.

PROMPT 1 — PLANNING

PASTE ALL OF IT

```
I have a website change request. Do NOT write any code yet.

REQUEST:
"""
[paste the whole request here - Asana text, what the
screenshots show, Loom transcript, follow-up messages]
"""

Before you plan anything:

1. Tell me what branch I'm on and whether it's clean and in
   sync with origin/main. If it isn't, stop and say so.

2. Read the actual files. Do not assume file names, component
   names, or where content lives - find them and cite real
   repo-relative paths and line numbers. If you tell me
   something exists, you must have opened it.

3. Tell me everywhere else this same content or data is
   rendered. Our content is usually shared between pages, so
   one change often needs to land in two or three places.
   Name every one you find.

4. If the request is ambiguous in a way that changes what you
   build, ask me now rather than guessing.

Then give me a plan with:

- Every file you'll add, edit or delete, with real paths
- What changes in each, in plain language
- Risks: what could break, what's hard to undo, and what you
  will NOT be able to check on my machine
- Exactly how you'll prove it works before I look
- Anything in the request you think is a bad idea, and why

Keep it short enough to read in two minutes. I'll approve or
correct it before you touch anything.
```

Expected result

A short written plan naming real file paths. No code written yet.

3.2 Read the plan properly before approving it

Check four things: does it do what was actually asked; is the file list proportionate (a one-line copy change touching nine files deserves a question); did it find the **other places** the same content appears; and did it ask you anything at all — zero questions on a vague request usually means it guessed.

Push back freely

If anything looks off, say so and ask for a revised plan. Rewriting a plan costs seconds. Rewriting the wrong code costs an afternoon.

Phase 4 — Build and check

4.1 Carry out the approved plan

Paste the whole prompt.

PROMPT 2 — EXECUTION

PASTE ALL OF IT

```
The plan is approved. Implement it now.

Rules:

- Build exactly what we agreed. If you find partway through
  that the plan was wrong, STOP and tell me. Don't improvise
  a different solution.
- Don't expand scope. If you spot something else worth
  fixing, list it at the end - don't fix it.
- Match the existing style of each file you touch: naming,
  formatting, and how heavily it's commented.

Before telling me it's done, verify it YOURSELF:

- Run: npx tsc --noEmit        (must report no errors)
- Run: npm run build           (must succeed)
- Start the site and LOOK at the change in a real browser.
  Confirm what I asked for is visibly there, and that what
  sits next to it still looks right.
- If it touches a form, an API route, or anything that can
  fail, test the failure path too - not just the happy path.

Then report:

- What you changed, file by file
- What you verified, with the ACTUAL output. Not "it works."
- What you could NOT check on my machine, and why, so I know
  what still needs testing on the preview link
- Then commit, with a message explaining WHY the change was
  made, not just what changed

If any check fails, say so plainly. Don't report success with
the failure buried at the bottom.
```

Expected result

A report naming what changed file by file, with actual command output — not “it works”.

4.2 Look at it yourself, locally

Start the dev server (your local test copy of the site), open the page you changed, and drag the window narrow to check mobile.

TERMINAL / CLI — TYPE IT YOURSELF

```
npm run dev

# then open http://localhost:3000
# Ctrl + C stops it
```

CLAUDE PROMPT — PASTE THIS IN

```
Start the dev server and show me the change. Screenshot the
section you edited at desktop width and again at 390px wide
so I can check mobile. Tell me if anything next to it looks
broken. Then give me the local URL so I can click around
myself.
```

Two things that fool everyone

Cannot see your change? Hard refresh with `Cmd + Shift + R` before assuming it failed. Browsers cache hard. Still nothing? Restart the dev server (`npm run dev`) and check you are on your branch with `git branch --show-current`, in that order.

Villa photos blank or green? Expected when `.env.local` is missing or incomplete. Listing photos and prices come from the booking system. Not a bug you caused — do not report it as one.

Expected result

Your change is visible at both widths and nothing beside it looks broken.

Phase 5 — Put it on a preview link

A real URL running real data, that is not the live site. You do not need a Netlify account and you do not run a deploy command — Netlify is already watching these repositories.

5.1 Push the branch and open a pull request

Uploading your branch to GitHub (a push) does not create a preview on its own. Opening the pull request is what triggers it.

POINT & CLICK — GITHUB, AFTER ONE PUSH

```
1.  Push the branch:
      git push -u origin fix/terms-page-update

2.  Open the repo on github.com. It offers a
      Compare & pull request  button. Click it, then
      Create pull request.

3.  Wait for the Deploy Preview check, then open the URL.

Do NOT merge yet. The pull request stays open until 6.2.
```

CLAUDE PROMPT — PASTE THIS IN

```
The change is finished and committed. Put it on a preview
link for review:

1. Push my current branch to origin and set upstream.
2. Give me the URL to open a pull request for this branch.
   Do not merge it.
3. Once I tell you the pull request is open, find the
   Netlify Deploy Preview URL for it.
4. Then fetch that URL and confirm my change is actually
   present on it, so I don't send the team a link to the
   wrong build.
```

If you ever use the Netlify CLI directly

You should not need to. But if you do, **never add `--prod`.** `netlify deploy --prod` publishes straight to the live site and skips every approval below. If you are unsure, do not run it — open a pull request instead.

Expected result

A **Deploy Preview** check appears on the pull request, then a URL of the form `deploy-preview-<number>--<site>.netlify.app`.

5.2 Confirm you are looking at YOUR version

Open the Deploy Preview URL and check the change is really there before showing anyone.

The most common mistake on this whole list

People check the **live site**, see the old version, and report the change as broken — or see something unrelated and report it as fixed. The live site does not contain your work yet. Only the Deploy Preview URL does. Check the address bar every single time.

Works locally but not on the preview?

Usually a setting (environment variable) missing in Netlify. Your computer reads `.env.local`; the preview reads Netlify's own settings. Ask a developer in the **Web Team** Google Chat group.

Expected result

The address bar shows `deploy-preview-<number>`, not the live domain.

Phase 6 — Approve and go live

6.1 Send the preview link to the right approver

Comment on the Asana task with the Deploy Preview URL, a one-line summary of what changed, and anything you could not verify yourself. Tag the approver from the table below.

IF THE CHANGE IS	TAG	EXAMPLES
Web, design, or anything touching how the site is built	**AnhTuan**	Layout, components, a new page, styling, anything in the code
An account or marketing task on a client site	**Makenna**	Client-supplied copy, a listing description, a campaign landing page
Organization-level, or it commits HGM to something	**Gillian**	Pricing, terms, legal or policy text, anything on HGM's own site

Route by what the change is, not who asked for it

A copy tweak an account manager requested is still a client-copy change, and goes to Makenna. When two rows apply, take the lower one: anything that commits HGM goes to Gillian.

If you are the approver

You cannot approve your own change. If AnhTuan made the edit, it goes to Gillian. The point of this task is a second pair of eyes, and self-approval is not that.

Expected result

An explicit yes. Wait for it — silence is not approval.

6.2 Merge the pull request, then clean up

The pull request has been open since 5.1. Open it on github.com and click **Merge**.

POINT & CLICK, THEN TERMINAL — AFTER MERGING ON GITHUB

```
git checkout main
git pull
```

CLAUDE PROMPT — PASTE THIS IN

```
The approver said yes. Help me finish:

1. Confirm my branch is still in sync, and that nothing new
   has landed on main that would conflict.
2. Give me the pull request URL for this branch so I can
   merge it on GitHub.

After I tell you the merge is done:

3. Switch me back to main and pull the latest.
4. Fetch the live site and confirm my change is actually
   there now.
```

Expected result

Netlify publishes the change to the live site automatically in 2–3 minutes.

6.3 Verify live, then close the task

Open the live site in a private / incognito window, confirm the change, then comment on the Asana task that it is live and mark it complete.

Why incognito

Your normal browser may serve you a cached copy of the old page, and extensions can hide things that work fine for everyone else. Incognito is closer to what a real guest sees. If a form or section is still blank, try a different network too: DNS filters block form and marketing tools, so it can be invisible to you and fine for everyone else.

Expected result

The change is visible to a logged-out visitor.

04 EXCEPTIONS AND EDGE CASES

Sites with extra checks

Some repositories also define `npm run lint` and `npm run typecheck`. Run those too where they exist, in addition to the two checks in Prompt 2.

The credential is missing

If `.env.local` is not in 1Password, get it from a developer and then ask them to store it properly. Do not keep a private copy as the only copy.

You need to undo everything

Not yet saved to git (committed): `git checkout .` throws away your edits. Committed but not merged: leave the branch behind with `git checkout main`. Nothing on a branch can affect the live site.

Near a usage limit

Run the mechanical tasks (2.1, 4.2, 6.2) yourself and save the remaining budget for task 4.1, which cannot be done another way.

If you are stuck

Ask Claude first, with Prompt 3 below, instead of guessing at a cause. If it is still unclear — changes you do not recognise, a missing credential, a Netlify setting — ask a developer (AnhTuan, Leshan, Brandon or Kyle) in the **Web Team** Google Chat group. Who approves a change is routed at task 6.1.

PROMPT 3 — DEBUGGING

USE INSTEAD OF GUESSING

```
Something isn't working. Before you change any code:

WHAT I EXPECTED:
[describe it]

WHAT ACTUALLY HAPPENS:
[describe it, and paste any error text exactly]

WHERE I SAW IT:
[local / preview URL / live site - paste the exact URL]

Please:

1. Confirm which version I'm actually looking at, so we don't
   debug the wrong build.
2. Find the real cause by reading the code and testing it.
   Don't guess. If you have a theory, prove it before acting
   on it.
3. Tell me the cause in plain language before you fix
   anything, and say how confident you are.

If it turns out I'm mistaken about the problem, tell me
directly.
```

05 FINAL CHECKLIST BEFORE YOU MERGE

- [ ] The request is fully done — every part of it, not just the easy parts
- [ ] `npx tsc --noEmit` reports no errors
- [ ] `npm run build` succeeds
- [ ] You looked at it locally, wide window and narrow
- [ ] It is on a branch, not on `main`
- [ ] A Deploy Preview URL exists and you confirmed your change is on it
- [ ] The right approver has said yes, explicitly
- [ ] Anything unverifiable is written in the Asana comment
- [ ] You checked the live site in incognito after merging

06 RELATED DOCUMENTS

- HGM-SOP-WEB-001 — Store, share, and update client .env files in 1Password
- HGM-SOP-OPS-001 — Produce and publish an HGM SOP
- Commands here are for HGM's Next.js client sites on Netlify. If a step turns out to be wrong or unclear, say so — a runbook nobody corrects stops being true.

07 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE SUMMARY
2026-09-01	1.0	Kyle Zinger	Initial release. Converted from the Website Edit Runbook; approval routing added and the preview step changed from a manual Netlify CLI deploy to the automatic pull-request Deploy Preview.
2026-09-11	1.1	Kyle Zinger	Hand-built task picker removed; the renderers now draw it from meta.tasks. No change to the procedure.
2026-09-11	1.2	Kyle Zinger	Plain-language sweep of task titles and instructions; six terms added to Definitions (repository, git and commit, push and merge, terminal, dev server, environment variable). No change to the procedure.
2026-09-12	1.3	Kyle Zinger	Route cards reframed by place of work (Claude Code in VS Code, Terminal in VS Code, Buttons and menus in VS Code) and reordered to match; comparison table columns reordered the same way. Portal page: skip-ahead card moved under the SOP details grid, side menu enlarged and its height bug fixed, section chip row restored in the header on every width, eased in-page scrolling, phase tools as pill buttons with Reset progress alongside. No change to the procedure.
2026-09-12	1.4	Kyle Zinger	Route selection reframed as a platform choice — VS Code (prompts, terminal and buttons), Terminal (commands only), Claude Desktop app (prompts only). Route cards, comparison table and portal selector updated to match; note added that the claude.ai website cannot run the prompts. No change to the procedure.
2026-10-15	1.5	Kyle Zinger	Loom review layout (house style v1.9). Roles, definitions and troubleshooting removed: terms are explained where first used, and each fix sits in the task it belongs to. Standing rules shortened; prerequisites collapse; an If you are stuck note; a copy-to-Claude button on the final checklist; links to other SOPs open in the portal. The platform cards and comparison table are gone: everyone works in VS Code, where every route runs.

REMEMBER

"Nothing on a branch can reach the live site until someone deliberately merges it. Everything else here is recoverable."

Questions?
Web Team Google Chat group